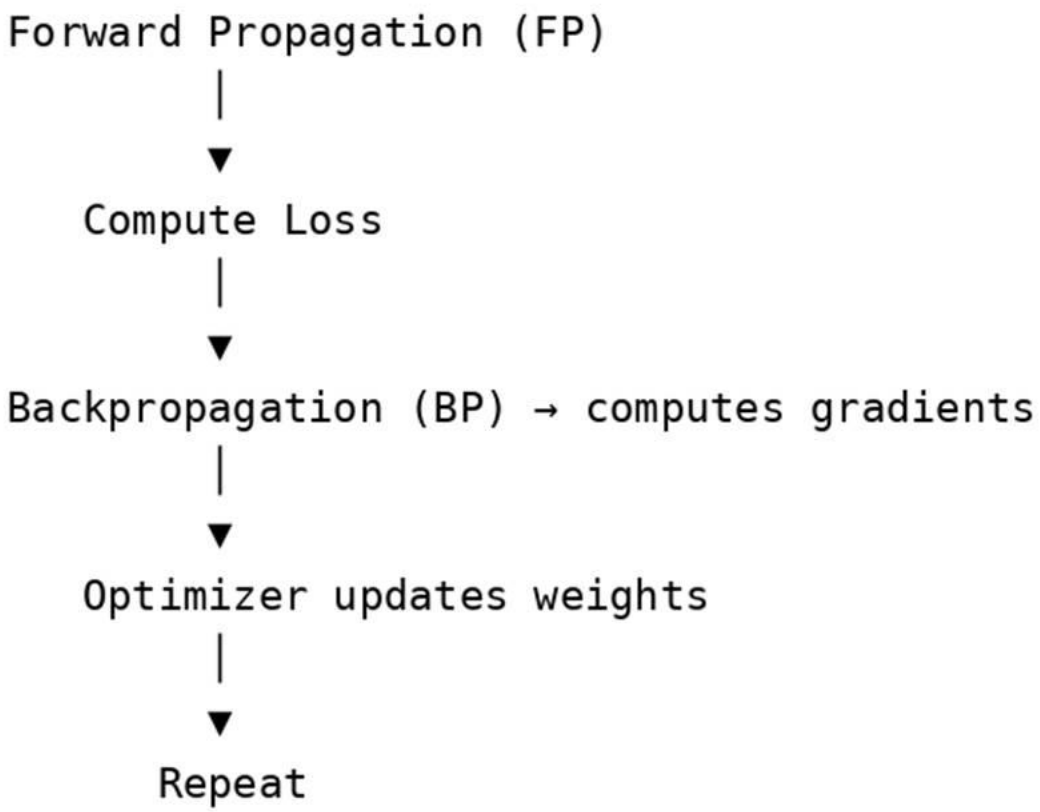
Optimizers in Neural Networks

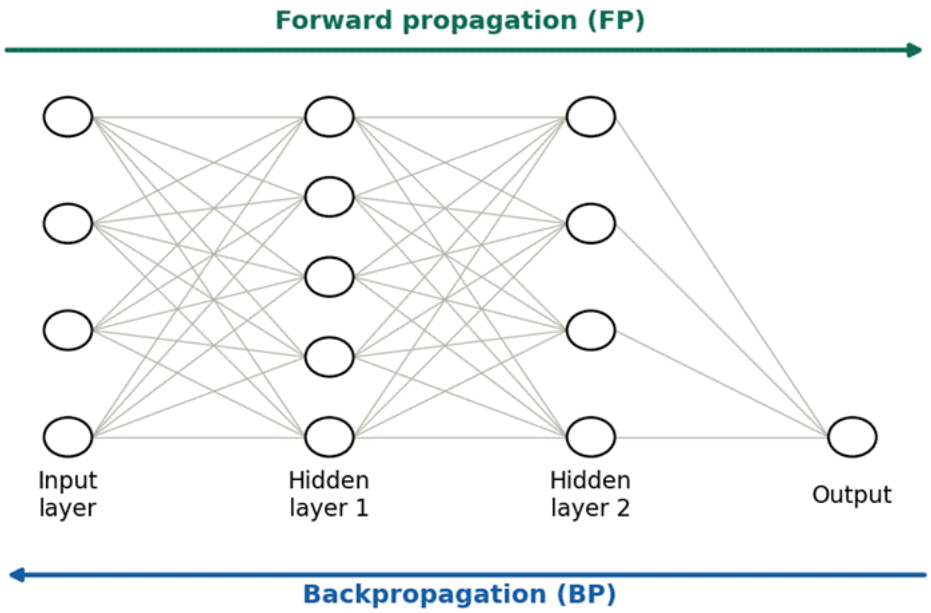
Training a neural network means finding the weight values that make its predictions as close as possible to the true targets. **Optimizers** are the algorithms that decide how the weights get updated at every step of training, using the gradients produced by backpropagation.

1. What is an Optimizer?

An optimizer is the component of the training loop that takes the gradients of the loss function (computed during backpropagation) and uses them to update the network's weights and biases.

Every training loop repeats the same cycle:





During **forward propagation**, the input passes through the layers to produce a prediction. The loss compares this prediction to the actual target:

$$Loss = y - \hat{y}$$

During **backpropagation**, this error is propagated backward through the network to compute how much each weight contributed to it — the gradient of the loss with respect to every weight. The **optimizer** then decides exactly how each weight should change based on that gradient.

2. Why do we need Optimizers?

Without an optimizer, a network would have no systematic way to improve. Gradients tell us the direction in which the loss increases fastest, but the network still needs a rule for:

- **How big a step to take** in the opposite direction (the learning rate)
- **How much data to look at** before making an update (all of it, one record, or a batch)
- **How to keep training stable** without overshooting or getting stuck

Optimizers answer these questions. Choosing the right optimizer directly affects:

- How fast the model converges
- Whether training is stable or noisy
- Whether the model gets stuck in a poor local minimum
- How much memory and compute each training step needs

3. The core update rule

Every gradient-based optimizer follows the same basic update rule:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$$

where:

- W_{old} = current weight value
- η = learning rate, controlling the step size
- $\frac{\partial L}{\partial W_{old}}$ = gradient of the loss with respect to the weight
- W_{new} = updated weight value

What differs between **Gradient Descent**, **Stochastic Gradient Descent**, and **Mini-batch Stochastic Gradient Descent** is not this formula — it's **how much data is used to compute the gradient before each update.**

4. Gradient Descent (GD)

4.1 What it is

In plain **Gradient Descent** (also called Batch Gradient Descent), the entire training dataset is passed through the network in a single forward and backward pass, and only **one** weight update happens using the average gradient over all records.

$$Gradient Descent \rightarrow \text{Complete dataset is used in a single iteration.}$$

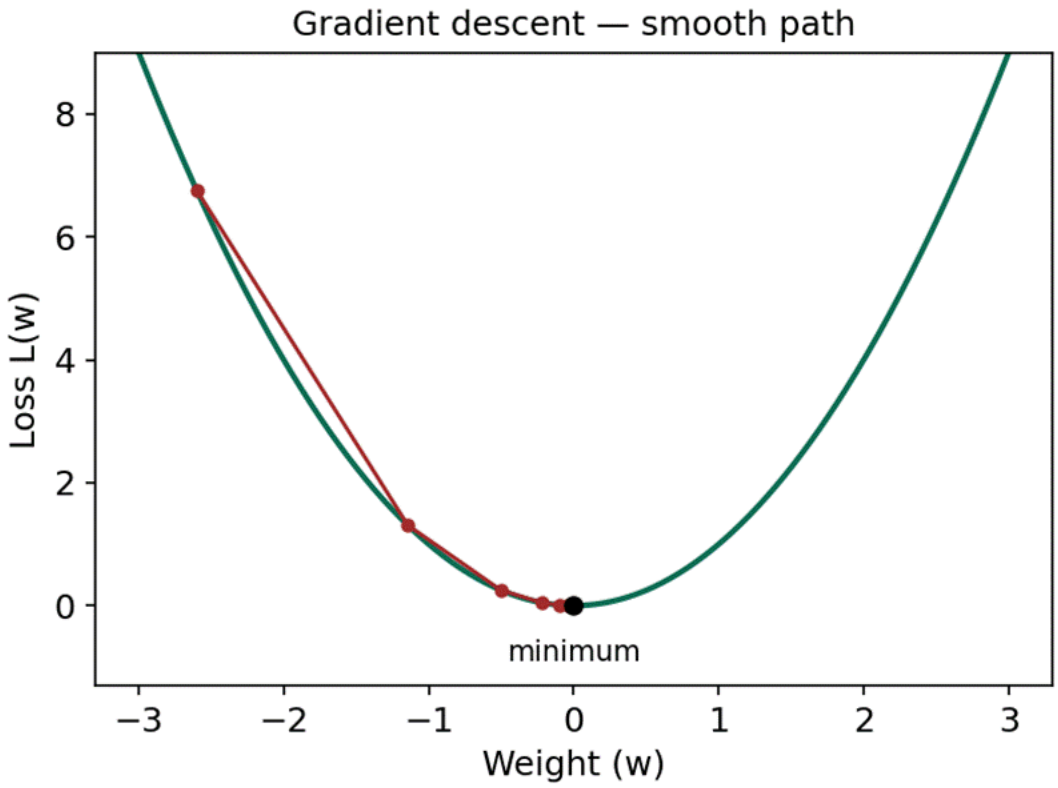
4.2 Iterations and epochs

If we have 10,000 training records:

$$10,000 records \rightarrow 1 iteration \rightarrow 1 epoch$$

So for Gradient Descent:

$$\boxed{\text{Iterations per epoch} = 1}$$



The path taken toward the minimum is smooth, because the gradient is computed from the full dataset at every step — there's no randomness in the direction.

4.3 Advantages

- Produces a smooth, stable path toward the minimum
- Gradient estimate is accurate since it uses the whole dataset
- Converges reliably for convex loss surfaces
- Easy to reason about and debug

4.4 Drawbacks

- Very slow for large datasets — an entire pass over the data is needed for just one update
- Requires the whole dataset to fit in memory at once
- Cannot be used for online/streaming learning (new data arriving continuously)
- Can get stuck in local minima or saddle points, since there is no noise to help escape them

5. Stochastic Gradient Descent (SGD)

5.1 What it is

Stochastic Gradient Descent updates the weights after looking at just **one training record at a time**. Instead of averaging the gradient over the whole dataset, it computes the gradient from a single example and updates immediately.

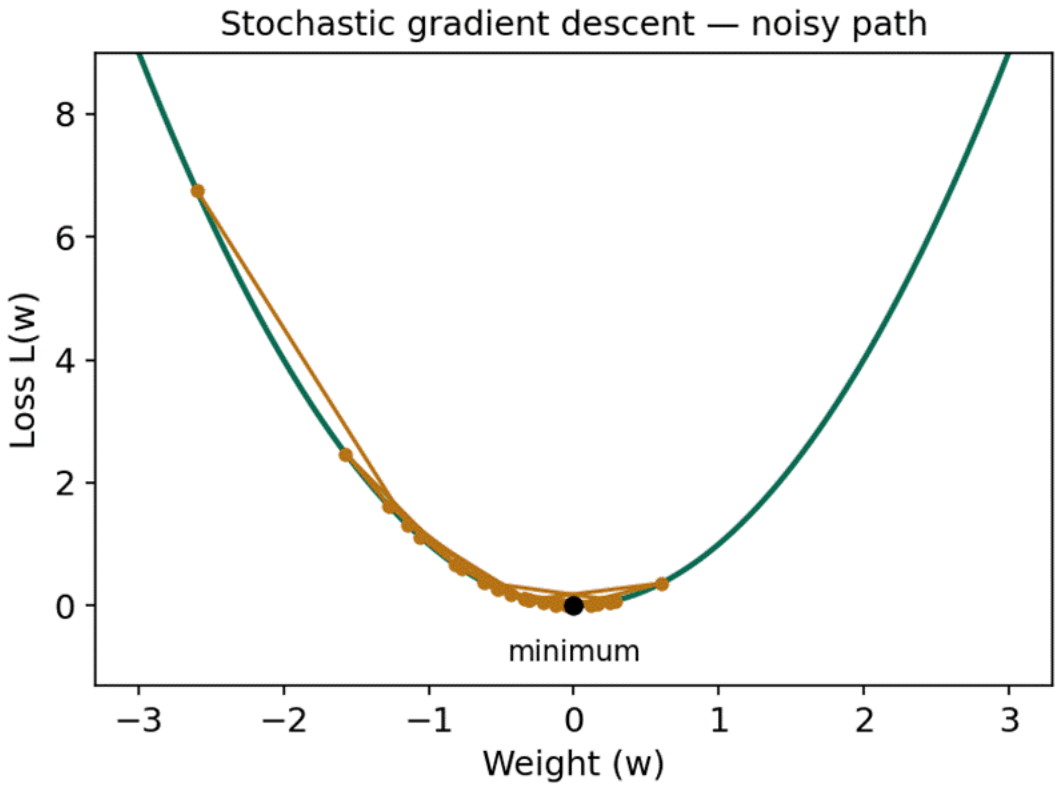
5.2 Iterations and epochs

If we have 10,000 training records:

$$1 Epoch \rightarrow Iterations = 10,000 (no.\ of records)$$

So for SGD:

$$\text{Iterations per epoch} = N \quad (\text{total number of records})$$



Because each update is based on only one noisy sample, the path to the minimum zig-zags a lot instead of moving smoothly.

5.3 Advantages

- Much faster updates — weights change after every single record
- Works well for very large datasets and online/streaming learning
- Needs far less memory since only one record is processed at a time
- The noise in the updates can actually help the model escape shallow local minima

5.4 Drawbacks

- The convergence path is very noisy and can oscillate around the minimum instead of settling precisely on it
- Because updates happen so frequently, the per-record overhead (looping, computing gradients repeatedly) adds up
- Requires a carefully tuned (often decaying) learning rate to converge well

- Less stable than Gradient Descent — harder to get consistent runs

6. Mini-batch Stochastic Gradient Descent

6.1 What it is

Mini-batch SGD is the middle ground between Gradient Descent and pure SGD. Instead of using the whole dataset or a single record, it splits the data into small **batches** and updates the weights once per batch.

If we have 10,000 records, for every epoch, data is fed in batches.

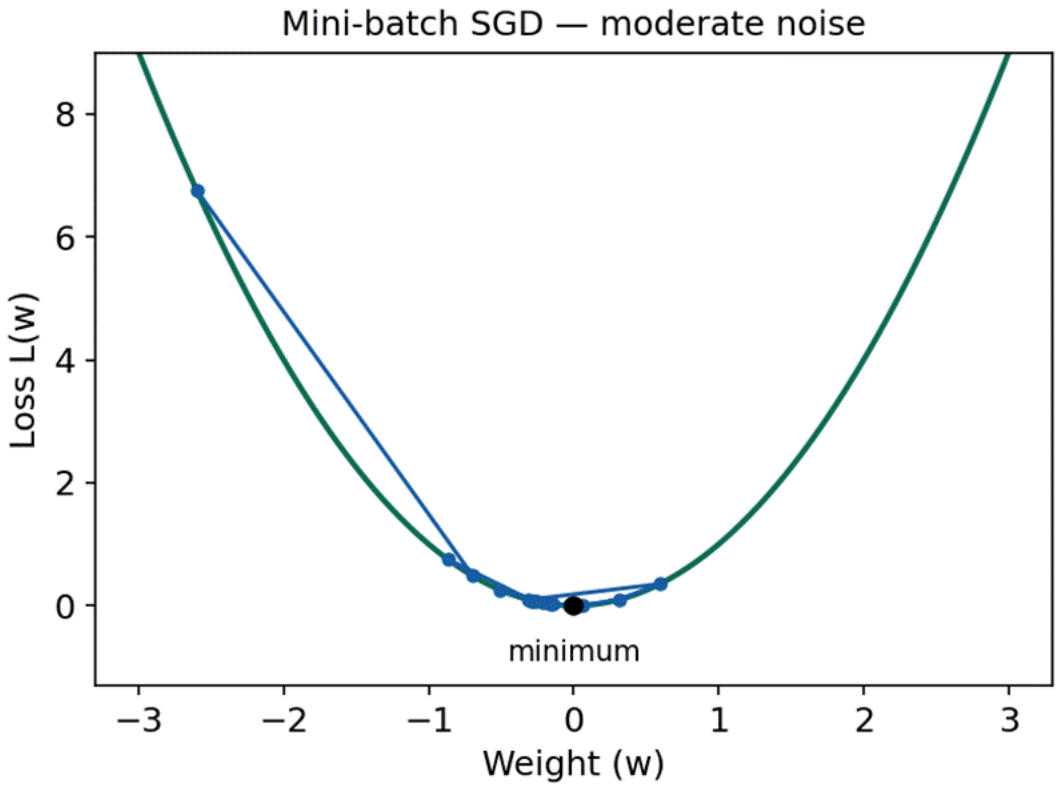
$$Batchsize = 1000$$

6.2 Iterations and epochs

$$Iterations = \frac{10,000}{Batchsize} = \frac{10,000}{1,000} = 10$$

So for Mini-batch SGD:

$$\text{Iterations per epoch} = \frac{N}{\text{batch size}}$$



The path is noisier than plain Gradient Descent, but far smoother than pure SGD, since each update is based on an averaged gradient over a batch of records rather than just one.

6.3 Advantages

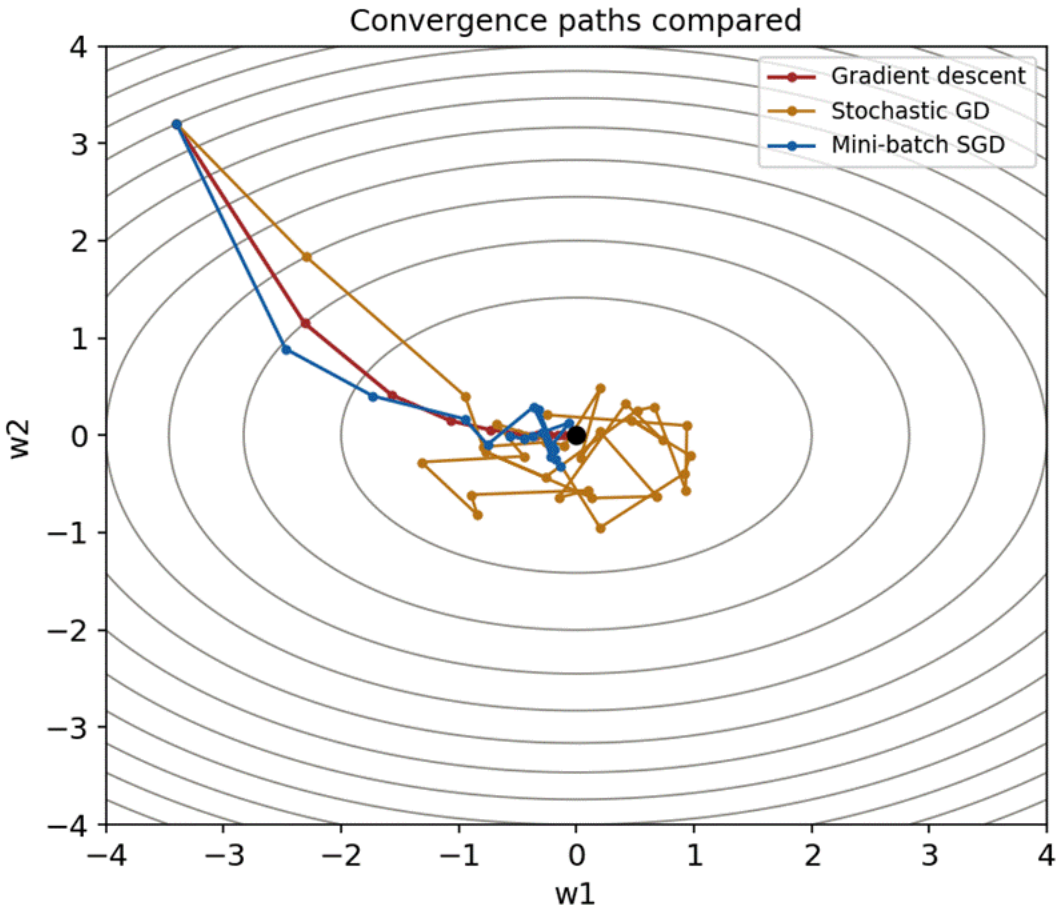
- A good balance of speed and stability — faster than GD, more stable than SGD
- Efficient on GPUs/hardware, since batched operations are vectorized
- Reduced variance in updates compared to SGD, while still retaining enough noise to help escape poor local minima
- Most widely used approach in practice for training deep networks

6.4 Drawbacks

- Batch size becomes an additional hyperparameter that needs tuning
- Still noisier than full-batch Gradient Descent
- Very large batch sizes reduce the benefit of noise and can approach GD's slowness; very small batch sizes approach SGD's instability

- Requires enough memory to hold one batch at a time (though far less than the full dataset)

7. Comparing all three paths together



On a contour plot of the loss surface, the three optimizers behave very differently on their way to the minimum:

- **Gradient Descent** (red) takes the most direct, smooth route.
- **Mini-batch SGD** (blue) follows a similar overall direction with some wobble.
- **Stochastic GD** (orange) zig-zags the most, but still works its way toward the minimum over many small steps.

8. Side-by-side comparison

Optimizer	Data used per update	Iterations per epoch	Convergence path	Memory usage
Gradient Descent	Entire dataset	1	Smooth	High
Stochastic Gradient Descent	1 record	N (number of records)	Very noisy	Very low
Mini-batch SGD	A batch of records	N / batch size	Moderately noisy	Moderate

9. Training loop summary for each optimizer

For 10,000 training records:

```
Gradient Descent
Epoch   <—   FP —►
        <— BP —    1 iteration
```

```
Stochastic Gradient Descent
Epoch   <—   FP —►
        <— BP —    Iterations = number of records = 10,000
```

```
Mini-batch Stochastic Gradient Descent
Epoch   <—   FP —►
        <— BP —    Iterations = 10,000 / batch size = 10,000 / 1,000 = 10
```

10. Key takeaway

All three optimizers use the exact same weight-update formula:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$$

The difference is purely **how much data feeds into each gradient calculation** before that formula is applied:

- **Gradient Descent** → all the data, once per epoch → smooth but slow
- **Stochastic Gradient Descent** → one record at a time → fast but noisy
- **Mini-batch SGD** → small batches at a time → the practical default that balances both